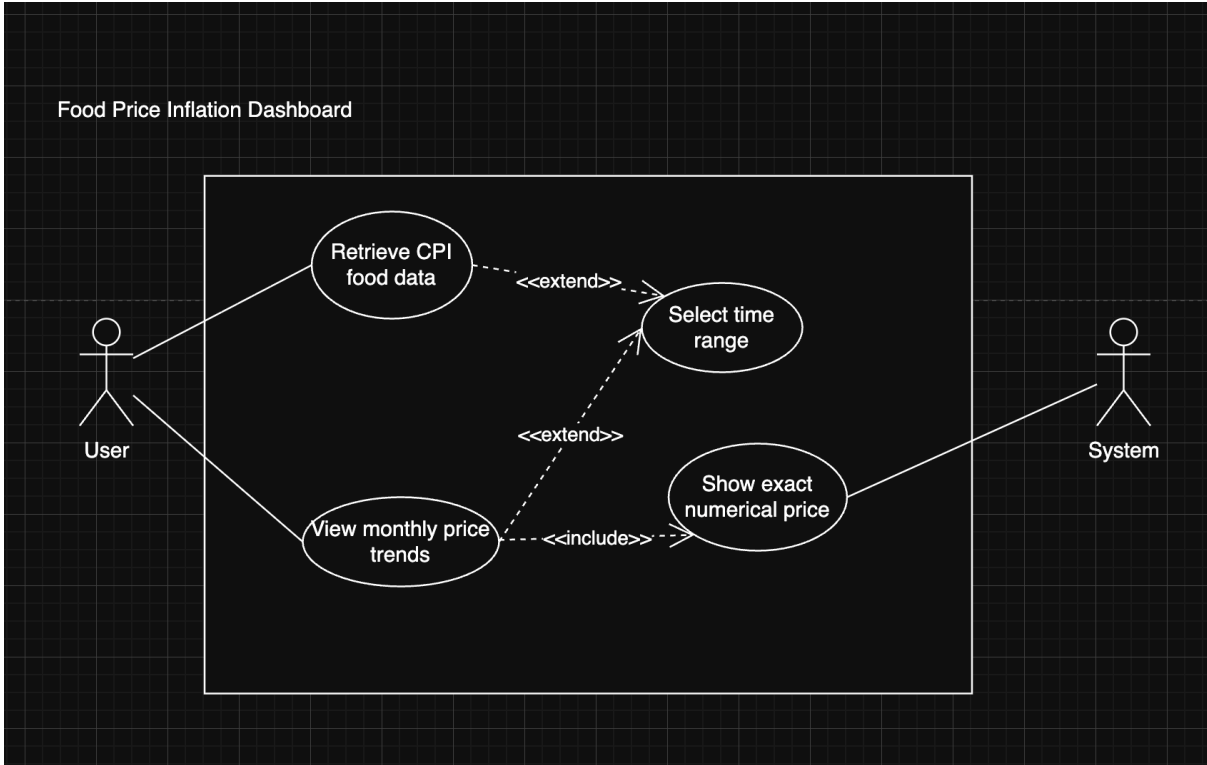


1. Food Price Inflation Dashboard

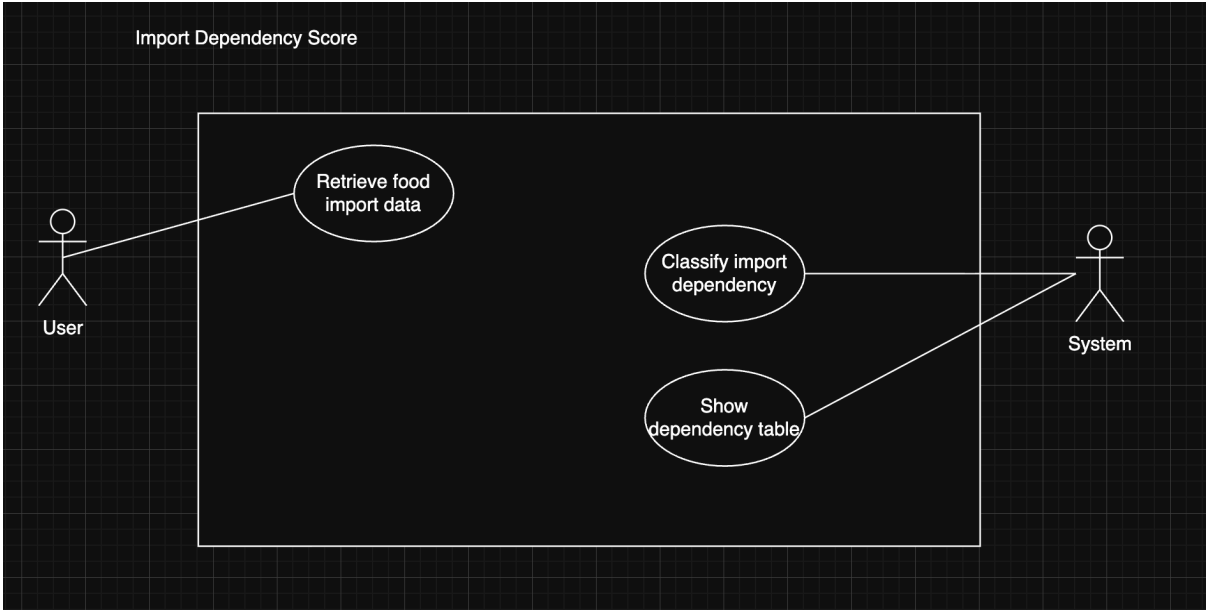


Use Case ID:	Use Case 1		
Use Case Name:	View Food Price Trends		
Created By:	Li Shuhui	Last Updated By:	Li Shuhui
Date Created:	2025.9.5	Date Last Updated:	2025.9.5

Actor:	-User (Student, Young Working Adult, Household) -System
Description:	The Food Price Inflation Dashboard enables users to track changes in food prices over time. It retrieves CPI data, generates line charts for monthly prices, and displays exact numerical values in SGD, helping users understand inflation trends for different food items.
Preconditions:	CPI dataset is available and up to date.
Postconditions:	The user leaves the current page or switches to another module of the system.
Priority:	Medium
Frequency of Use:	Occasionally
Flow of Events:	1.User navigates to the Food Price Inflation Dashboard. 2.User selects food items to be inquired. 3.System retrieves CPI (Consumer Price Index) data for selected food items. 4.User selects a time range (e.g., last 6 months). 5.System generates a line chart for monthly prices.

	6.System displays numerical price values in SGD (2 decimals).
Alternative Flows:	1.If user selects a different food item -System asks for selecting a time range. -System returns to step 5 2.If user adjusts the time range - The system updates the chart accordingly.
Exceptions:	1.If CPI data is missing for a month -System shows “Data unavailable” -System continues with available months. 2.If system response > 2s -System displays a loading message until the chart is ready.
Includes:	Viewing monthly price trends includes showing exact numerical price
Special Requirements:	Price values must be within ± 0.05 SGD of CPI dataset.
Assumptions:	-CPI dataset is always available and up-to-date. -Users have stable internet access to view dynamic charts.
Notes and Issues:	-Historical data gaps may cause incomplete charts. -Large time ranges may increase response time slightly

2. Import Dependency Score

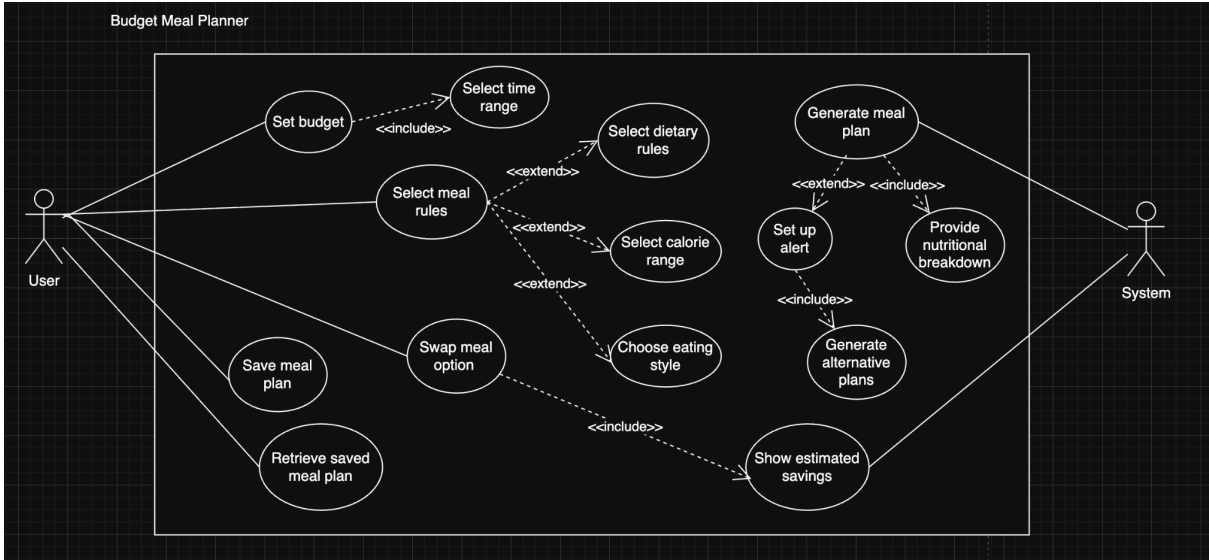


Use Case ID:	Use Case 2		
Use Case Name:	View Import Dependency of Foods		
Created By:	Li Shuhui	Last Updated By:	Li Shuhui
Date Created:	2025.9.5	Date Last Updated:	2025.9.5

Actor:	-User (Student, Young Working Adult, Household) -System
Description:	The Import Dependency Score feature provides users with information about how reliant different foods are on imports.
Preconditions:	Import dataset is available and up to date.
Postconditions:	The user leaves the current page or switches to another module of the system.
Priority:	Medium
Frequency of Use:	Occasionally
Flow of Events:	1.User navigates to Import Dependency Score. 2.User input a food type 3.System retrieves import data by food type. 4.System calculates import percentage. 5.System classifies dependency as Low (0–50%), Medium (51–80%), or High (81–100%). 6.System displays a table with food items, import percentage, and risk level.
Alternative Flows:	If User filters results by category (e.g., grains, meat, vegetables). -System filters the dataset to display only food items within the selected category.

	-System returns to step 4.
Exceptions:	1.If dataset is incomplete -System marks missing foods as "Data not available". 2.If system response > 2s -System displays a loading message until the table is ready.
Includes:	N/A
Special Requirements:	Accuracy of import percentage must be within $\pm 0.5\%$ of source data
Assumptions:	The user has internet access.
Notes and Issues:	Some foods may fall into ambiguous categories and require manual adjustment.

3. Budget Meal Planner

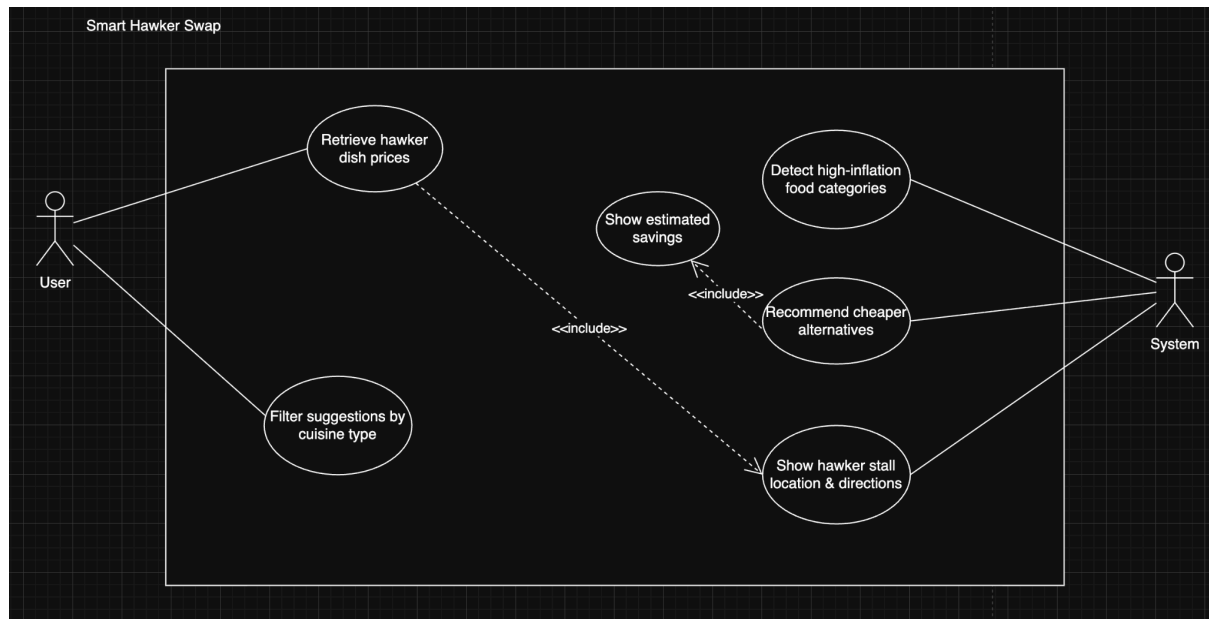


Use Case ID:	Use Case 3		
Use Case Name:	Generate Budget Meal Plan		
Created By:	Li Shuhui	Last Updated By:	Li Shuhui
Date Created:	2025.9.5	Date Last Updated:	2025.9.5

Actor:	-User (Student, Young Working Adult, Household) -System
Description:	The Budget Meal Planner helps users design affordable daily or weekly meal plans.
Preconditions:	The user has logged into the website.
Postconditions:	The user leaves the current page or switches to another module of the system.
Priority:	High
Frequency of Use:	Frequently
Flow of Events:	1.User navigates to the Budget Meal Planner. 2.User sets daily/weekly budget in SGD. 3.User selects dietary rules (e.g., halal, vegetarian). 4.User selects calorie range. 5.User chooses “Eat out” or “Cook at home”. 6.System generates 2-3 meal plans with itemized costs, food options, total cost and nutrition breakdown.

Alternative Flows:	1.If user clicks “Swap” - The system provides alternative meal plans with a cheaper or alternative option that still fits the meal rules. -System shows estimated savings. 2.If user saves plan -System stores the plan under the user’s ID. 3.If user retrieves saved plan -System displays all the saved plans under the user’s ID. 4.If System provide a plan exceeding the budget or the saved plans exceeding the current budget after the user changes the budget -System sets up an alert for the user. -System provides alternative plans to keep total cost under budget if possible.
Exceptions:	1.If no valid meal plan fits budget -System suggests closest alternatives and highlights cost overrun. 2.If system response > 5s -System displays a loading message until the plan is ready.
Includes:	-Setting a budget includes selecting a time range. -Swapping meal option includes showing estimated savings. -Generating meal plan includes providing nutritional breakdown. -Setting up alerts includes generating alternative plans.
Special Requirements:	Price estimates must be within $\pm 10\%$ of CPI-adjusted baseline.
Assumptions:	-Users provide reasonable dietary and calorie inputs. -Food pricing data is current and covers common meal ingredients.
Notes and Issues:	-Substitutions must always include a clear explanation for transparency. -Some dietary restrictions (e.g., vegan + halal combined) may limit available meal options.

4. Smart Hawker Swap

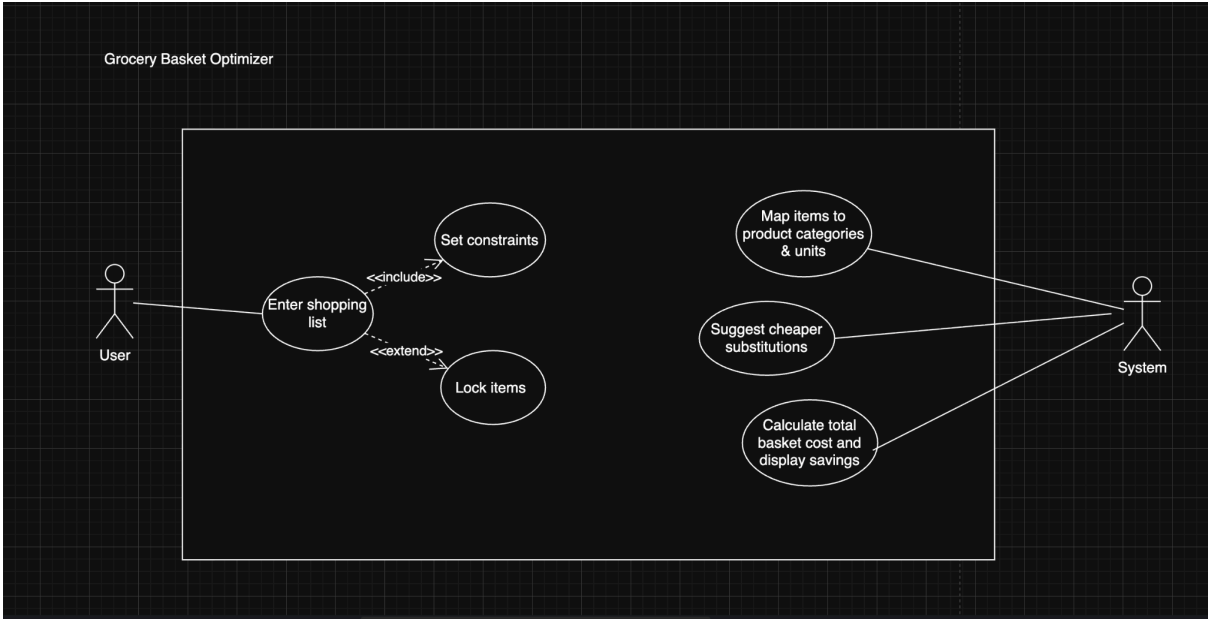


Use Case ID:	Use Case 4		
Use Case Name:	Find Cheaper Hawker Alternatives		
Created By:	Li Shuhui	Last Updated By:	Li Shuhui
Date Created:	2025.9.5	Date Last Updated:	2025.9.5

Actor:	-User (Student, Young Working Adult, Household) -System
Description:	The Smart Hawker Swap suggests cheaper hawker food alternatives to help users cope with rising dish prices.
Preconditions:	-User has logged into the website. -User has allowed location access .
Postconditions:	-User leaves the current page or switches to another module of the system.
Priority:	High
Frequency of Use:	Frequently
Flow of Events:	1.User navigates to Smart Hawker Swap. 2.System displays average dish prices nearby from CPI/NEA datasets. 3.System displays food categories with high inflation. 4.User choose one dish in the interface. 5.System shows location details and directions to stalls. 6.System recommends cheaper alternatives within 3 km radius if possible. 7.System displays estimated savings.
Alternative Flows:	1.If user selects a cuisine filter -System provides suggestions by cuisine type.

	-System returns to step 5. 2.If user requests route map -System opens location in map view.
Exceptions:	1.If no hawker stalls found nearby -System displays “No nearby alternatives found”. 2.If GPS data missing -System asks for manual location input. 3.If dataset incomplete -System marks missing dishes as “Data unavailable”. 4.If system response > 5s -System displays a loading message until it is ready.
Includes:	-Retrieving hawker dish prices includes showing hawker stall location & directions. -Recommending cheaper alternatives includes showing estimated savings.
Special Requirements:	Location accuracy must be within 300 m.
Assumptions:	The user has internet access.
Notes and Issues:	GPS signals may be inaccurate indoors, affecting nearby stall suggestions

5. Grocery Basket Optimizer



Use Case ID:	Use Case 5		
Use Case Name:	Optimize Grocery Basket		
Created By:	Li Shuhui	Last Updated By:	Li Shuhui
Date Created:	2025.9.5	Date Last Updated:	2025.9.7

Actor:	-User (Student, Young Working Adult, Household) -System
Description:	The Grocery Basket Optimizer allows users to enter a shopping list and receive an optimized basket that fits their budget and dietary preferences.
Preconditions:	-User has logged into the website. -User has a shopping list ready to enter.
Postconditions:	User leaves the current page or switches to another module of the system.
Priority:	High
Frequency of Use:	Frequently.
Flow of Events:	1.User enters a shopping list in free-text form. 2.System parses the list and maps each item to a product category and unit. 3.User specifies constraints such as budget and dietary tags. 4.User optionally locks specific items to prevent substitution. 5.System checks available items and suggests cheaper substitutions at category level. 6.System calculates the total basket cost and displays percentage savings.

Alternative Flows:	1.If user adjusts budget or dietary constraints -System re-runs optimization and updates basket results. 2.User decides not to apply any substitutions -System shows original basket cost without savings.
Exceptions:	1.If system cannot map an item to any category -System flags it as “Unrecognized item” and prompts the user to clarify. 2.If no substitutions available for a given item -System retains the original item and marks it as “No alternative found”. 3.If dataset temporarily unavailable -System shows “Data not available, please try again later.” 4.If system response > 5s -System displays a loading message until it is ready.
Includes:	Entering a shopping list includes setting constraints.
Special Requirements:	All substitutions and savings calculations must be accurate within $\pm 10\%$ of actual prices.
Assumptions:	The user has internet access.
Notes and Issues:	-Some uncommon items may not be mapped correctly and require user clarification. -Savings calculations are based on average prices; actual store prices may vary.